

Data Schema (ERD)

AI-Powered Content Operations Platform — Skip Studio

Prepared for	Skip Studio (fictional client — Exology Pioneer Program, Week 4 Final Project)
Prepared by	Amr Hassan Safieddine Nagy
Date	August 24, 2026
Status	Draft — for review

1 What this describes

The things the product stores: clients and the market they operate in, the people who work on them, the brand and compliance rules that govern their content, the regional calendar their content plans around, the campaigns and every multi-form deliverable they produce, the two-stage approval history behind each one, the Instagram account each client is connected through, the performance data that comes back once something is live, and the requests and comments a client raises directly. Built for full traceability: every scheduled item can be traced back to the brief it came from, the client and market it belongs to, the brand guide version it was grounded in, the rule it cited, and the exact approval history that cleared it to publish.

2 Entities

People & access

► User

Field	Notes
<code>user_id</code> PK	
<code>name</code>	
<code>email</code>	unique; the login identifier
<code>password_hash</code> , nullable	null until the account is activated — a client contact created by their account manager has no password yet. Hashed, never reversible; never leaves the server.
<code>user_type</code>	<code>staff</code> <code>client_contact</code>
<code>is_agency_admin</code>	staff only; assigns roles on client records
<code>must_change_password</code>	true after an OTP redemption, forcing a password set before anything else is reachable
<code>status</code>	<code>invited</code> <code>active</code> <code>disabled</code> — <code>invited</code> means created but never signed in
<code>last_login_at</code> , nullable	

Every role signs in; there is no anonymous route. A client contact is created by their account manager rather than self-registering, so the account exists before its owner ever sees it — hence `password_hash` nullable and `status = invited` as the starting state.

► **LoginOtp** — the one-time code that activates an invited account

Field	Notes
<code>otp_id</code> PK	
<code>user_id</code> FK → User	
<code>code_hash</code>	hashed like a password — a leaked table must not hand over working codes
<code>expires_at</code>	short-lived
<code>consumed_at</code> , nullable	set on redemption; a code works exactly once
<code>created_by_id</code> FK → User	the account manager who issued it, for the audit trail

The code is displayed **on screen to the account manager** at creation and never stored in readable form. There is no email integration in this version (PRD §4), so the AM passes it on by whatever channel they already use with that client. It grants nothing by itself: redeeming it only unlocks setting a password.

► **Session** — a signed-in user

Field	Notes
<code>session_id</code> PK	the opaque token, stored hashed
<code>user_id</code> FK → User	
<code>expires_at</code> , <code>created_at</code>	
<code>revoked_at</code> , nullable	set on sign-out

Server-side sessions rather than a self-describing token, so a sign-out or a disabled account takes effect immediately. Every request derives the acting user from here — never from a query parameter or header the browser controls.

► **ClientAssignment** — per-client team, beyond the default account manager

Field	Notes
<code>assignment_id</code> PK	
<code>client_id</code> FK → Client, <code>user_id</code> FK → User	
<code>role_on_client</code>	<code>content_lead</code> (replaces account manager as reviewer) <code>content_creator</code> <code>client_approver</code>

A `client_approver` User may have exactly one `ClientAssignment` row, ever. Staff roles are intentionally many-to-many with Client; a client contact is not.

Enforced by a **partial** unique index — `(user_id) WHERE role_on_client = 'client_approver'` — not by a plain unique on `user_id`, which would also forbid a content creator working across several accounts. The model-level `@unique([client_id, user_id, role_on_client])` does not express this rule on its own: it permits the same contact to approve for CL-101 and CL-102, since those rows differ by `client_id`. That is the cross-client hole the invariant exists to close, so the index is declared in raw SQL (Prisma's schema language cannot express a filtered index) and `lib/domain/clientContactInvariant.ts` enforces the same rule with a readable error.

Clients & markets

► Client — *the client roster*

Field	Notes
client_id PK	
name , industry	
status	active inactive — gates whether any content may be produced
tier	informational only, no behavior tied to it
channels	list
account_manager_id FK → User, nullable	default internal reviewer for this client; null for former clients with no active owner (e.g. CL-109)
active_brand_guide_id FK → BrandGuideVersion, nullable	null = no governing guide on file — the common case: only 8 of the 150 seeded clients have one
sensitive_sector	derived from industry (healthcare / financial / government)

A client's markets are **not** a column here — see **ClientMarket**. A client operates in one or more markets, chosen by the account manager at creation.

► Market — *seeded with two rows*

Field	Notes
market_id PK	
name	"Egypt" "Saudi Arabia"
country_code	EG SA
calendar_system	informational — e.g. "gregorian_and_hijri"

A real table rather than a hardcoded enum, so a third market is a data insert, not a migration. Egypt is the market the seeded roster and brand guides are written for (NileFit's audience is "Cairo and Alexandria"; agency Clause 1.3's worked example is "Egypt's leading"). Saudi Arabia is seeded alongside it so the multi-market path is demonstrable rather than theoretical.

► ClientMarket — *which markets a client operates in*

Field	Notes
client_market_id PK	
client_id FK → Client, market_id FK → Market	unique together

Every client has at least one row. The account manager picks one or both at client creation, and may change the set later. This is what makes a market a many-to-many rather than a column, without a schema change when a third market is added.

Brand governance

► BrandGuideVersion

Field	Notes
<code>brand_guide_version_id</code> PK	
<code>client_id</code> FK → Client	
<code>version_number</code>	
<code>status</code>	<code>draft</code> <code>pending_client_approval</code> <code>active</code> <code>superseded</code>
<code>created_by_id</code> FK, <code>client_approved_by_id</code> FK, nullable	
<code>created_at</code> , <code>approved_at</code>	

Only one version per client is ever active. Past versions stay superseded, never deleted —

`ContentItem.grounded_brand_guide_version_id` freezes the version a draft was written under, and deleting one would turn every such reference into a dangling claim about what was checked.

The lifecycle is small and enforced in `lib/domain/brandGuides.ts`:

```
draft --(AM submits)--> pending_client_approval --(client approves)--> active \-(client declines)--> draft
```

A declined version returns to `draft` rather than to a terminal `declined` status, for the same reason a declined `ContentItem` resets to `drafted`: it needs fixing by whoever wrote it, and re-enters the client's queue only when someone deliberately resubmits. There is no `declined` value in the status enum at all.

Approval is the only thing that moves `Client.active_brand_guide_id`. Superseding the outgoing version, promoting the incoming one, and repointing that column happen in one transaction, so the client never has two active guides or none.

`retrievalScope.getGuidelinesForClient` reads that column and nothing else, which is what makes "which rules govern this client right now" have exactly one answer.

► GuidelineClause — the agency standards and every client's brand rules

Field	Notes
<code>clause_id</code> PK	
<code>source_type</code>	<code>agency</code> <code>brand</code>
<code>brand_guide_version_id</code> FK, nullable	null for agency clauses — global, unversioned
<code>clause_code</code> , <code>title</code> , <code>text</code>	e.g. <code>0.6</code> , <code>CR.4</code> , <code>NB.4</code>

Calendar

► Occasion — regional and local calendar entries, scoped to one market

Field	Notes
<code>occasion_id</code> PK	
<code>market_id</code> FK → Market	
<code>name</code> , <code>category</code>	e.g. "Ramadan"; religious national seasonal retail
<code>shared_key</code> , nullable	set when the same observance exists in more than one market — e.g. <code>ramadan</code> , <code>eid_al_fitr</code> . Lets the engine recognise Egypt's and Saudi Arabia's Ramadan rows as one occasion and produce a single market-neutral item, instead of

Field	Notes
	two near-duplicates. National days have no <code>shared_key</code> and stay market-specific.
<code>date_type</code>	<code>fixed_gregorian</code> <code>hijri_based</code>
<code>month</code> , <code>day</code>	used only when <code>date_type</code> = <code>fixed_gregorian</code>

► **OccasionDate** — *resolved per-year dates for occasions whose Gregorian date moves*

Field	Notes
<code>occasion_date_id</code> PK	
<code>occasion_id</code> FK → Occasion	only populated for <code>hijri_based</code> occasions
<code>year</code> , <code>gregorian_date</code>	the resolved date for that year
<code>source</code>	<code>seeded</code> — hand-maintained, not calculated

Campaigns & content

► **Campaign** — *the brief, and the campaign it becomes*

Field	Notes
<code>campaign_id</code> PK	
<code>client_id</code> FK → Client	
<code>title</code> , <code>objective</code> , <code>audience</code> , <code>channels</code> , <code>raw_brief_text</code>	as submitted
<code>related_occasion_id</code> FK → Occasion, nullable	set when built around a specific upcoming occasion
<code>submitted_by_id</code> FK → User	
<code>status</code>	<code>received</code> <code>info_requested</code> <code>in_progress</code> <code>complete</code>
<code>override_attempt_detected</code>	true if the brief tried to skip or fake approval
<code>compliance_review_required</code>	true for sensitive-sector clients

► **ContentItem** — *one row per drafted deliverable*

Field	Notes
<code>content_item_id</code> PK	
<code>campaign_id</code> FK → Campaign	
<code>content_form</code>	post image video reel photoshoot email blog_post ad_copy hashtag_set cta creative_prompt
<code>platform</code>	instagram tiktok facebook linkedin email
<code>content_body</code>	current draft text; visual/video forms carry content via linked MediaAsset rows instead
<code>market_id</code> FK → Market, nullable	null = market-neutral (evergreen or shared-occasion) item, produced once; set = item written for that one market's occasion and scheduled against that market's date. Must be a market the client operates in.
<code>scheduled_date</code>	target/proposed publish date — distinct from the scheduled status

Field	Notes
status	drafted in_refinement pending_internal_review internal_approved pending_client_review client_approved scheduled declined flagged publishing published publish_failed
flagged_clause_id FK, nullable	primary rule broken, if flagged
parent_content_item_id FK, nullable	self-reference for A/B variants
grounded_brand_guide_version_id FK	version active at draft time — frozen even if the guide is edited later
created_at , updated_at	

► **ContentItemCitation**

Field	Notes
citation_id PK	
content_item_id FK, clause_id FK	

Every drafted item can be grounded in more than one clause — an agency clause and a brand rule together.

► **MediaAsset** — *generated or uploaded images and video*

Field	Notes
media_asset_id PK	
content_item_id FK → ContentItem	
asset_type	image video
generation_source	ai_generated uploaded — a photoshoot's output is always uploaded
storage_url , format	
created_by_id FK → User	

► **ReferenceAttachment** — *reference material a creator supplies when prompting generation or regeneration of one item*

Field	Notes
attachment_id PK	
content_item_id FK → ContentItem	the specific deliverable being generated or regenerated
uploaded_by_id FK → User	must resolve to role_on_client = content_creator
file_type	image pdf doc
storage_url	
instruction	optional text accompanying the file, e.g. "match this angle"
created_at	

Image references are passed as vision context; PDF/doc references are text-extracted as context. Rows accumulate across regenerations rather than being overwritten. Carries no authority over compliance, same as brief wording and comments.

Review & compliance

- **Approval** — *one record per decision, per item, per stage*

Field	Notes
<code>approval_id</code> PK	
<code>content_item_id</code> FK → ContentItem	
<code>stage</code>	<code>internal</code> <code>client</code>
<code>decision</code>	<code>approve</code> <code>decline</code> — an approval that requests a change is recorded as decline
<code>comment</code>	required if decline, optional if approve
<code>decided_by_id</code> FK → User, <code>decided_at</code>	
<code>bulk_action_id</code> , nullable	groups rows created by one "approve whole plan" click

The gate reads the most recent row per `(content_item_id, stage)` — not whether an approval exists anywhere in history. This is what lets a reviewer or client decline something they already approved. Decline is available at `pending_internal_review` / `pending_client_review`, and remains available at `internal_approved`, `client_approved`, and `scheduled`, for either the internal reviewer/content lead or the client. Decline stops applying once `publishing` or `published`.

- **Flag** — *anything routed to a human instead of drafted or scheduled*

Field	Notes
<code>flag_id</code> PK	
<code>campaign_id</code> FK, <code>content_item_id</code> FK, nullable	null content_item_id = brief-level flag
<code>clause_id</code> FK, nullable	the rule broken — null for the governance types, which breach a role boundary rather than a clause
<code>flag_type</code>	content: <code>brand_violation</code> <code>compliance_violation</code> <code>unknown_client</code> <code>inactive_client</code> — raised by the engine, about the work governance: <code>cross_client_data</code> <code>approval_override_attempt</code> <code>role_boundary_violation</code> <code>off_task_generation</code> <code>approval_churn</code> — raised anywhere, about a person's conduct, surfaced to the Agency Admin
<code>severity</code>	<code>high</code> (a real breach: override attempt, cross-client access, role violation) <code>medium</code> (off-task generation) <code>low</code> (approval churn — a process signal, not a rule breach). Lets the Admin queue rank breaches above noise.
<code>raised_against_id</code> FK → User, nullable	who did it, for the governance types. Null for engine-raised content flags, which are about a brief, not a person.
<code>resolved</code> , <code>resolution_notes</code> , <code>resolved_at</code>	

Two kinds of flag share one table because both are "route this to a human". They differ in who reads them: a content flag goes to the account manager handling that brief, a governance flag goes to the Agency Admin. `severity` and `raised_against_id` are what the Admin's misuse queue sorts and groups on.

`cross_client_data` is a tripwire, not a routine path: retrieval scoping makes cross-client access structurally impossible, so a row here means a real bug or a real attempt, and is always `high`.

► **AuditLog** — *append-only*

Field	Notes
audit_id PK	
entity_type , entity_id , action	created edited scheduled rescheduled deleted approved declined flag_raised flag_resolved published take_down
details , nullable	free-form JSON context for the action — what changed, and from what to what
performed_by_id FK → User, performed_at	

Publishing & performance

► **PlatformConnection** — *a client's connected Instagram account*

Field	Notes
platform_connection_id PK	
client_id FK → Client	
platform	instagram (enum kept open for future platforms)
access_token	encrypted at rest — never stored or logged in plaintext
platform_account_id , nullable	Instagram's own account id; safe to display to a client, unlike the token
token_expires_at , nullable	drives the expired status
status	connected expired disconnected
connected_by_id FK → User, connected_at	staff only

No privacy field — Instagram Professional accounts cannot be private, so there is nothing to model.

► **MetricSnapshot** — *time-series, not a single mutable row*

Field	Notes
metric_snapshot_id PK	
content_item_id FK → ContentItem	
captured_at , metric_type , value	impressions reach likes comments shares saves

Client requests

► **PostRequest** — *the client-initiated calendar ask; carries no authority*

Field	Notes
post_request_id PK	
client_id FK, requested_by_id FK → User (client_contact)	
requested_date	the calendar day being asked for
related_content_item_id FK, nullable	set for "change/reschedule this existing post"
status	new under_review converted declined withdrawn

Field	Notes
linked_campaign_id FK, nullable	set once converted into a real brief

A request is editable and withdrawable **by the client who raised it, while it is still `new`** (PRD §6). The account manager takes it by moving it to `under_review`, and that is the moment the client's edit controls switch off — an explicit action rather than a side effect of the AM opening the page, so both sides can see exactly when the request stopped moving.

`withdrawn` is a distinct status rather than a reuse of `declined`, because the two are different facts: `declined` is the agency saying no, `withdrawn` is the client changing their mind. Collapsing them would make the AM's queue unable to tell the two apart. A withdrawn row is kept rather than deleted — its `Comment` thread is part of the client's conversation with their account manager.

► **Comment** — discussion thread, distinct from `Approval.comment`

Field	Notes
comment_id PK	
post_request_id FK, nullable	exactly one of these two is set
content_item_id FK, nullable	
author_id FK → User, body	

A Comment never triggers an approval reset — only a formal Approval decline or a deliberate edit does.

► **Conversation** — an ongoing exchange between one staff member and the engine, scoped to one client (Phase 14)

Field	Notes
conversation_id PK	
client_id FK → Client	the scope. Every retrieval, every grounding decision and every access check reads it — a conversation is <i>about</i> one client, and cannot drift to another
created_by_id FK → User, nullable	the creator or content lead who owns the thread
campaign_id FK → Campaign, nullable	set once the thread has said enough to produce work. Null while it is still being assembled
title , status	<code>active</code> <code>archived</code>

Why a `campaign_id` at all. Items must stay linked to the brief they came from (PRD §3), and a conversation is the brief in this path — assembled over turns rather than submitted at once. So when the exchange first has enough to work from, a real `Campaign` row is created and everything the thread produces hangs off it exactly as a submitted brief's items do. Downstream — the review queues, the gate, analytics — nothing can tell the two apart, which is the point.

Nullable until then, because a thread that has not yet named an objective is not yet a campaign, and writing an empty one would put half-formed rows in the account manager's queue.

► **ConversationTurn** — one message in a thread

Field	Notes
turn_id PK	
conversation_id FK → Conversation	
role	<code>creator</code> <code>assistant</code> <code>system</code>

Field	Notes
<code>body</code>	what was said
<code>flag_id</code> FK → Flag, nullable	set when this turn was refused, linking the Admin's queue row to the turn in its thread
<code>created_at</code>	ordering is by time; turns are never edited or deleted

The transcript is evidence, not a convenience. Turns are append-only for the same reason `AuditLog` is: the Admin's misuse queue judges conduct across a conversation, and a record someone can revise after the fact proves nothing. A refused turn is kept, with `flag_id` pointing at the reason — so the queue shows what was asked, in what context, rather than an isolated excerpt.

`ReferenceAttachment.turn_id` (nullable) puts an attached file on the turn it arrived on. The existing `content_item_id` stays, so material attached before Phase 14 is untouched.

3 PRD roles → schema representation

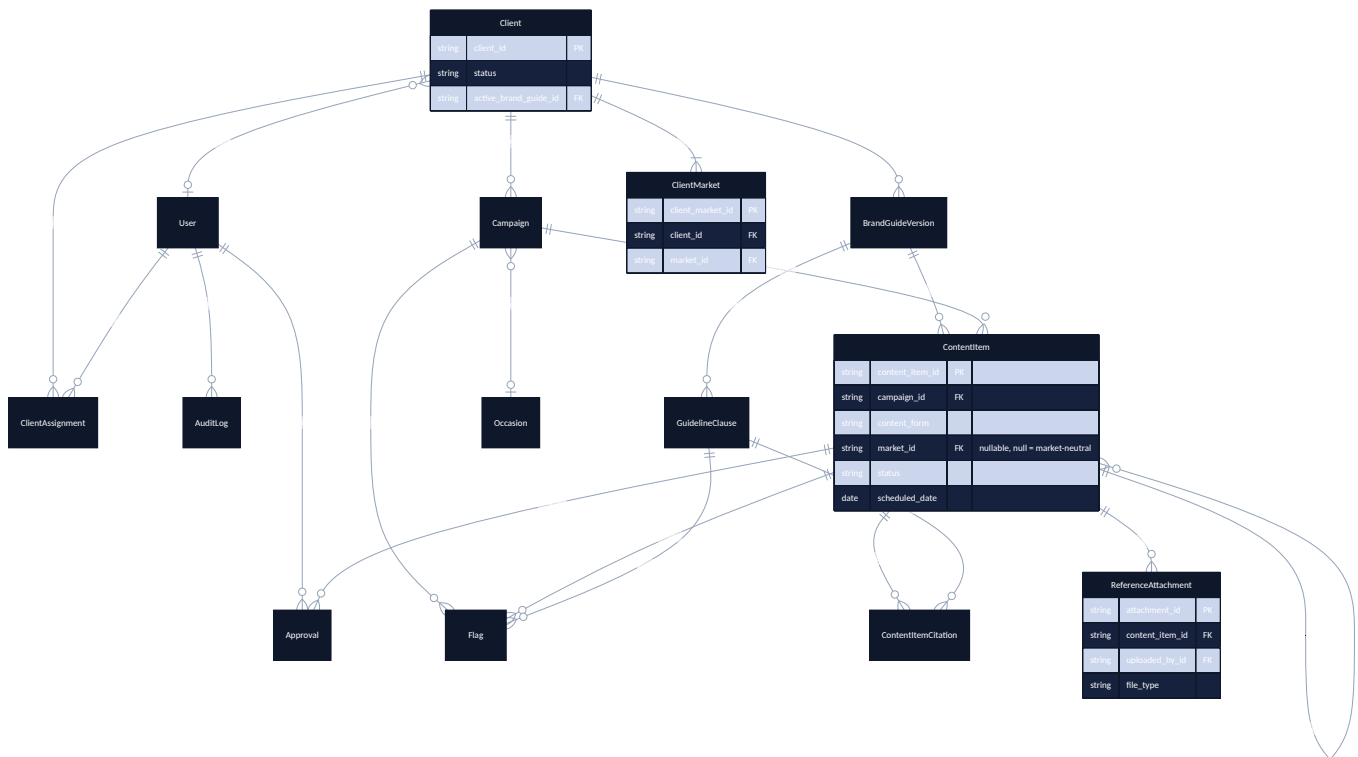
Five user types, none with a dedicated table — each is a `User` plus, for client-scoped roles, a `ClientAssignment` row. This is what lets Admin assign or reassign anyone by editing a row, with no separate user-management screen.

Role	user_type	role_on_client	Sees	Notes
Account Manager	<code>staff</code>	— (<code>Client.account_manager_id</code>)	Clients they manage	Submits briefs; is the internal reviewer unless a content lead is assigned; creates clients and their client contacts; manages PlatformConnection; converts PostRequests.
Content Creator (incl. Marketing Specialist)	<code>staff</code>	<code>content_creator</code>	Clients they are assigned to	Zero or more per client; originates content in a <code>Conversation</code> scoped to one of their clients; refines drafts pre-internal-review; attaches image/PDF/doc references when prompting generation or regeneration. May be handed an item by a lead via <code>ContentItem.assigned_to_id</code> .
Content Lead	<code>staff</code>	<code>content_lead</code>	All clients	Optional per client; prompts the engine in a <code>Conversation</code> as a creator does, and dispatches what it produces to a creator via <code>ContentItem.assigned_to_id</code> . Replaces the account manager as internal reviewer for that client, with the same late-revoke power. Reviews across accounts, so unlike the creator their view is not scoped by assignment.
Client	<code>client_contact</code>	<code>client_approver</code>	Their own client only	One or more per client. Created by their account manager and activated with a one-time code. Approves/declines via the same Approval table, stage = client. Sees their assigned account manager; raises PostRequests and Comments; views their own analytics.

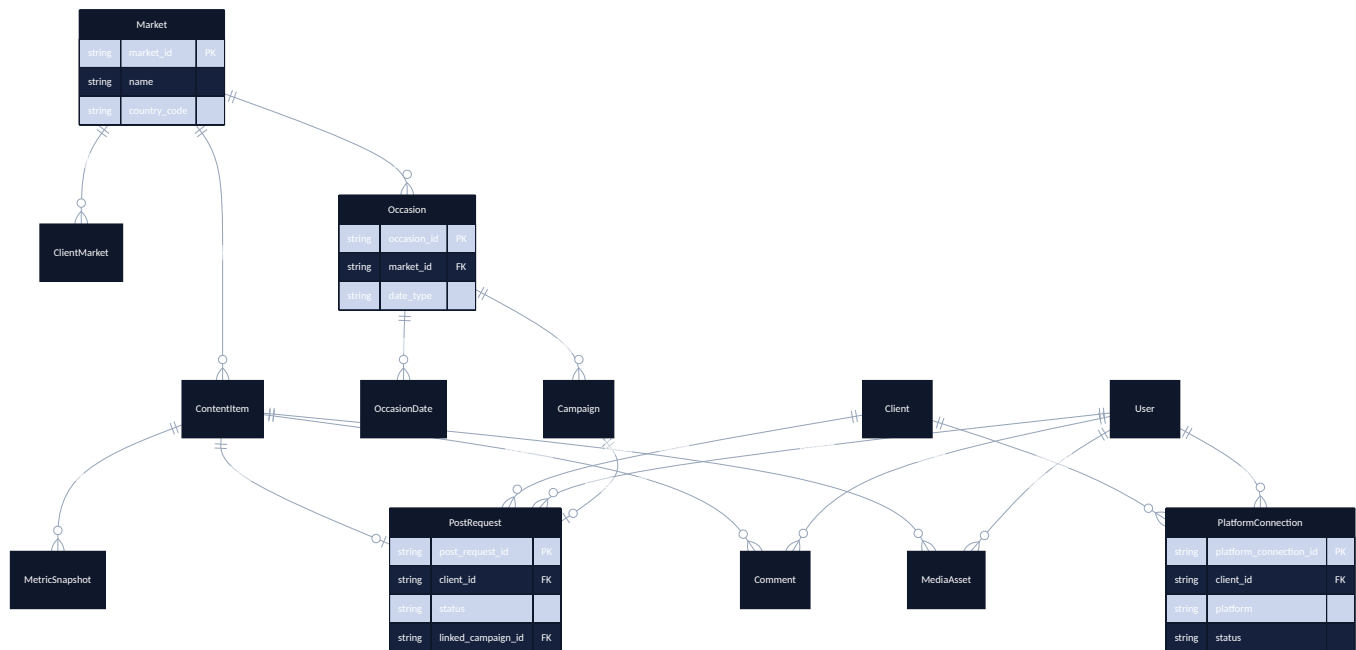
Role	user_type	role_on_client	Sees	Notes
Agency Admin	staff, admin=true	—	All clients	Not tied to any one client; edits account_manager_id on Client and rows in ClientAssignment directly. The accountability role: the only reader of the governance Flag queue, and the reader of conversation transcripts behind an off_task_generation flag. Since Phase 14 the account manager also holds client.assign_roles , so the admin's staffing power is a backstop rather than exclusive. Every assignment change either of them makes writes an AuditLog row naming them.

Two roles see every client — the Content Lead, because reviewing spans accounts, and the Agency Admin, because oversight is the job. The other three are scoped, and the scope is derived from the signed-in user on every request rather than from anything the browser supplies.

4 Relationship diagram — client, campaign & content



5 Relationship diagram — market, publishing & client requests



6 Traceability

Requirement	Satisfied by
Choose market — one or both of Egypt and Saudi Arabia	Market table seeded with two rows + ClientMarket join, set by the account manager at client creation
A dual-market client's plan covers both	ContentItem.market_id nullable — market-neutral items produced once, occasion-specific items produced per market
Regional and local occasion calendar	Occasion + OccasionDate , scoped by market_id, Hijri-aware, shared_key collapsing observances common to both markets
Full multi-form content plan	ContentItem.content_form + MediaAsset
Schedules refer to upcoming events and occasions	resolve_calendar engine step + Campaign.related_occasion_id + scheduled_date
Publish to Instagram	PlatformConnection + Publishing Layer, scoped to one tester account
Performance analytics for Account Manager and Client	MetricSnapshot , time-series, role-scoped queries
Account Manager creates clients with market	Client create flow, market_id required
Client sees assigned account manager	Direct read of Client.account_manager_id
Client calendar request/schedule with comments	PostRequest + Comment , no authority over the gate
Edited approved or scheduled post resets approval	ContentItem.status reset on content or scheduled_date change
Client and reviewer can decline after approval or scheduling	Approval, gate reads most-recent-per-stage, symmetric for both actors
Nothing scheduled or published without both current approvals	Atomic gate re-check in the scheduler — dedicated race-condition test
A published post cannot be silently declined	status = published excluded from decline; separate take-down action

Requirement	Satisfied by
No client's content or guide visible to another client	Every content-bearing table scoped by <code>client_id</code> via <code>campaign_id</code> → Client, filtered by <code>visibleClientIds(user)</code> derived from the session
Creator can attach reference files for content generation	<code>ReferenceAttachment</code> , scoped to one <code>content_item_id</code> , creator-only, image/PDF/doc only; <code>turn_id</code> links it to the conversation turn it arrived on
Creators and the content lead produce content by prompting the model	<code>Conversation</code> + <code>ConversationTurn</code> , scoped by <code>client_id</code> ; items produced through the same engine and gate as a submitted brief
Misuse of the engine is visible to the Admin	<code>Flag.flag_type = off_task_generation</code> + <code>ConversationTurn.flag_id</code> — the queue row opens the thread around the refused turn
A lead can hand a produced item to a creator	<code>ContentItem.assigned_to_id</code> , nullable; dispatch only, grants no access and gates no approval
Every role signs in; a client contact is created for them	<code>User.email</code> / <code>password_hash</code> / <code>status</code> , <code>LoginOtp</code> , <code>Session</code> — account manager issues a one-time code, client redeems it and sets a password
Access differs by role	Client → own client; Account Manager → <code>account_manager_id</code> matches; Content Creator → <code>ClientAssignment</code> rows; Content Lead and Agency Admin → all clients